
Introduction

This project was completed using **Ubuntu Server 24.04.3 LTS** and **CentOS 9**. Before beginning the hardening process, I compared both operating systems with the versions used throughout the course in order to identify security-related differences, default configurations, and changes introduced in newer releases.

The hardening procedures were successfully applied to both operating systems. The course significantly strengthened the Linux security fundamentals I had acquired during the previous year and provided valuable hands-on experience with server hardening techniques.

For the final project, **Ubuntu Server 24.04.3 LTS** was selected as the primary operating system

General security:

```
itsafe@linuxsec:~$ sudo apt upgrade -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following upgrades have been deferred due to phasing:
  python3-software-properties software-properties-common
0 upgraded, 0 newly installed, 0 to remove and 2 not upgraded.
itsafe@linuxsec:~$ sudo apt list --upgradable
Listing... Done
python3-software-properties/noble-updates 0.99.49.4 all [upgradable from: 0.99.49.2]
software-properties-common/noble-updates 0.99.49.4 all [upgradable from: 0.99.49.2]
itsafe@linuxsec:~$
```

System Update

The hardening process began with a complete system update to ensure that all installed packages were running the latest available versions.

After upgrading the system, an additional verification was performed to confirm that no further package updates were available. This step ensured that the server was fully patched before implementing additional security controls.

Firewall Configuration (UFW)

As part of the system hardening process, the **Uncomplicated Firewall (UFW)** was enabled to provide basic network access control.

The following security policy was implemented:

- **Default Incoming Policy:** deny
- **Default Outgoing Policy:** allow

To support secure remote administration, **TCP port 22 (SSH)** was explicitly allowed while all other unsolicited inbound connections remained blocked by default.

The firewall configuration was verified using the following command:

```
sudo ufw status verbose
```

The output confirmed that **UFW was active** and that the configured security policies were correctly applied.

```
itsafe@linuxsec:~$ sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow 22/tcp
sudo ufw enable
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
Rules updated
Rules updated (v6)
Command may disrupt existing ssh connections. Proceed with operation (y|n)? Y
Firewall is active and enabled on system startup
itsafe@linuxsec:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
22/tcp ALLOW IN Anywhere
22/tcp (v6) ALLOW IN Anywhere (v6)

itsafe@linuxsec:~$
```

Advanced Kernel Hardening (sysctl)

Beyond the course requirements, additional kernel hardening measures were implemented using the **sysctl** framework. These settings provide enhanced protection against several common network-based attacks, including:

- SYN Flood attacks
- IP Spoofing
- ICMP Redirect attacks
- Source Routing attacks

The kernel configuration file was edited:

```
sudo nano /etc/sysctl.conf
```

The following security parameters were added:

TCP SYN Cookies

```
net.ipv4.tcp_syncookies = 1
```

Enables **TCP SYN Cookies**, providing protection against **SYN Flood** attacks. Instead of immediately allocating resources for every incoming TCP connection request, the kernel validates the client using a cryptographic challenge before establishing the connection.

Reverse Path Filtering (Anti-Spoofing)

```
net.ipv4.conf.all.rp_filter = 1
```

```
net.ipv4.conf.default.rp_filter = 1
```

Enables **Reverse Path Filtering (RPF)** to mitigate **IP Spoofing** attacks. Incoming packets with forged source IP addresses are discarded if the system determines that a reply would not be routed through the same network interface.

Disable ICMP Redirects

```
net.ipv4.conf.all.accept_redirects = 0
```

Disables acceptance of **ICMP Redirect** messages, preventing attackers from manipulating the system's routing table as part of **Man-in-the-Middle (MITM)** attacks.

Disable Source Routing

net.ipv4.conf.all.accept_source_route = 0
Disables **Source Routing**, a legacy IP feature that allows the sender to specify the network path of packets. Since this functionality is rarely used in modern environments and may be abused to bypass network security controls, it is considered a security best practice to disable it.

Configuration Notes

The /etc/sysctl.conf file includes many kernel security parameters that are commented out by default. As part of the hardening process, the relevant security settings were enabled and customized according to Linux security best practices.

```
#####  
# Additional settings - these settings can improve the network  
# security of the host and prevent against some network attacks  
# including spoofing attacks and man in the middle attacks through  
# redirection. Some network environments, however, require that these  
# settings are disabled so review and enable them as needed.  
#  
# Do not accept ICMP redirects (prevent MITM attacks)  
#net.ipv4.conf.all.accept_redirects = 0  
#net.ipv4.conf.default.accept_redirects = 0  
#_or_  
# Accept ICMP redirects only for gateways listed in our default  
# gateway list (enabled by default)  
# net.ipv4.conf.all.secure_redirects = 1  
#  
# Do not send ICMP redirects (we are not a router)  
#net.ipv4.conf.all.send_redirects = 0  
#  
# Log Martian Packets  
#net.ipv4.conf.all.log_martians = 1  
#  
#####
```

```
#####  
# Additional settings - these settings can improve the network  
# security of the host and prevent against some network attacks  
# including spoofing attacks and man in the middle attacks through  
# redirection. Some network environments, however, require that these  
# settings are disabled so review and enable them as needed.  
#  
# Do not accept ICMP redirects (prevent MITM attacks)  
net.ipv4.conf.all.accept_redirects = 0  
net.ipv4.conf.default.accept_redirects = 0  
#_or_  
# Accept ICMP redirects only for gateways listed in our default  
# gateway list (enabled by default)  
#net.ipv4.conf.all.secure_redirects = 1  
#  
# Do not send ICMP redirects (we are not a router)  
net.ipv4.conf.all.send_redirects = 0  
#  
# Log Martian Packets  
net.ipv4.conf.all.log_martians = 1  
#  
net.ipv4.conf.default.rp_filter = 1  
net.ipv4.conf.all.rp_filter = 1  
net.ipv4.tcp_syncookies = 1  
#####
```

Default Ubuntu Security Parameters

Ubuntu already enables several additional security-related kernel parameters by default, including:

net.ipv4.conf.all.log_martians = 1

Logs packets containing suspicious or invalid source addresses ("martian packets"), providing useful information for security monitoring and incident analysis.

net.ipv4.conf.all.send_redirects = 0

Prevents the server from sending **ICMP Redirect** messages to other hosts, ensuring that the system does not participate in modifying or influencing network routing decisions.

These default settings further strengthen the operating system's network security posture without requiring additional configuration.

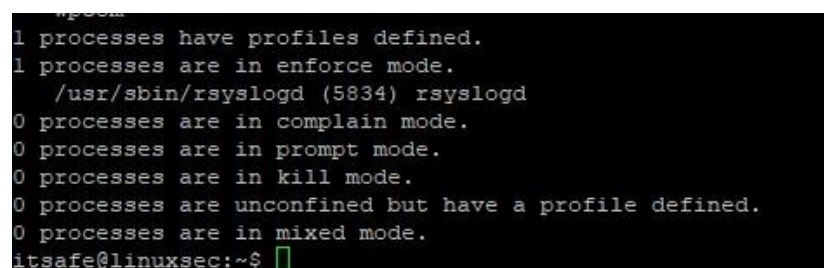
General Security Verification

After completing the kernel hardening configuration, the **AppArmor** Mandatory Access Control (MAC) framework was verified to ensure it was active and enforcing security policies.

The following command was executed:

```
sudo aa-status
```

The output confirmed that **AppArmor** was running correctly and that the appropriate security profiles were loaded and enforcing restrictions on supported applications.



```
1 processes have profiles defined.
1 processes are in enforce mode.
  /usr/sbin/rsyslogd (5834) rsyslogd
0 processes are in complain mode.
0 processes are in prompt mode.
0 processes are in kill mode.
0 processes are unconfined but have a profile defined.
0 processes are in mixed mode.
itsafe@linuxsec:~$
```

The verification results indicated that **AppArmor** was fully operational and actively enforcing security policies.

Key findings included:

- All loaded security profiles were running in **enforce** mode.
- The **rsyslog** service was protected by an active AppArmor profile in accordance with the system's security policy.
- No profiles were found operating in **complain** mode or **mixed mode**, indicating that the enforced security policies were consistently applied across the protected services.

These results confirm that the Mandatory Access Control (MAC) framework was functioning as expected and providing an additional layer of protection beyond traditional Linux file permissions.

Physical Security

Bootloader (GRUB) Hardening

To improve the physical security of the server, the **GRUB bootloader** was hardened to prevent unauthorized users with physical access from modifying the boot process or bypassing operating system security controls.

As part of this process, a **PBKDF2-encrypted password** was generated and configured for the GRUB bootloader. Storing the password in encrypted form ensures that the original password cannot be recovered directly from the configuration files.

Protecting the bootloader helps mitigate attacks in which an attacker attempts to:

- Boot the system into **single-user** or **recovery mode**.

- Modify kernel boot parameters.
- Gain unauthorized root access by bypassing normal authentication mechanisms.
- Disable security controls before the operating system fully loads.

```
itsafe@linuxsec:~$ grub-mkpasswd-pbkdf2
Enter password:
Reenter password:
PBKDF2 hash of your password is grub.pbkdf2.sha512.10000.5A0639FA90D5FDD6F950D59227F7F42
ADC98BCB98DEE1827081DC3010A941EF7C21F422B59BBA0232F99BF7E7EB2E003CF41DDA97416
```

/etc/grub.d/40_custom updating the file:

```
GNU nano 7.2
#!/bin/sh
exec tail -n +3 $0
# This file provides an easy way to add custom menu entries.  Simply type the
# menu entries you want to add after this comment.  Be careful not to change
# the 'exec tail' line above.
set superusers="admin"
password_pbkdf2 admin grub.pbkdf2.sha512.10000.5A0639FA90D5FDD6F950D59227F7F42
```

Grub update

```
itsafe@linuxsec:~$ sudo update-grub
Sourcing file `/etc/default/grub'
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-6.8.0-101-generic
Found initrd image: /boot/initrd.img-6.8.0-101-generic
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done
itsafe@linuxsec:~$
```

:Checking

```
Enter username:
admin
Enter password:
-
```

Additional Physical Security Recommendations

Although not implemented as part of this project, the following measures are considered security best practices for production environments:

- Configure a **BIOS/UEFI administrator password** to prevent unauthorized firmware modifications.
- Disable booting from external media (USB/DVD) to reduce the risk of offline attacks or unauthorized operating system access.
- Restrict changes to the system boot order to authorized administrators only.

These controls complement GRUB protection by providing an additional layer of physical security.

Password Policy Enforcement (PAM)

The next phase of the hardening process focused on enforcing a strong password policy using the **Pluggable Authentication Modules (PAM)** framework.

Starting with **Ubuntu Server 24.04.3 LTS**, the legacy **cracklib** module has been replaced by **libpam-pwquality**, which provides enhanced password complexity validation and more flexible password policy enforcement.

The required package was installed using:

```
sudo apt install libpam-pwquality
```

The password quality policy was then configured by editing the following configuration file:

```
sudo nano /etc/security/pwquality.conf
```

This file defines the system-wide password complexity requirements, including parameters such as minimum password length, character diversity, repeated characters, dictionary word detection, and other security constraints designed to reduce the risk of weak or easily guessable passwords.

```
# Cannot be set to lower value
minlen = 10
#
# The maximum credit for having
# it is the minimum number of
dcredit = -1
#
# The maximum credit for having
# If less than 0 it is the minimum
# password.
ucredit = -1
#
# The maximum credit for having
# If less than 0 it is the minimum
# password.
lcredit = -1
#
# The maximum credit for having
# If less than 0 it is the minimum
# password.
ocredit = -1
#
# The minimum number of required
# password (digits, uppercase
# minclass = 0
```

```
# applications.
# The new password is rejected if it fails the check and the value is not 0.
# enforcing = 1
#
# Path to the cracklib dictionaries. Default is to use the cracklib default.
# dictpath =
#
# Prompt user at most N times before returning with error. The default is 1.
# retry = 3
#
# Enforces pwquality checks on the root user password.
# Enabled if the option is present.
```

The file update successfully

```

itsafe@linuxsec:~$ passwd
Changing password for itsafe.
Current password:
New password:
BAD PASSWORD: The password contains less than 1 uppercase letters
New password:
BAD PASSWORD: The password contains less than 1 digits
New password:
BAD PASSWORD: The password contains less than 1 non-alphanumeric characters
passwd: Have exhausted maximum number of retries for service
passwd: password unchanged
itsafe@linuxsec:~$ passwd
Changing password for itsafe.
Current password:
passwd: Authentication token manipulation error
passwd: password unchanged
itsafe@linuxsec:~$ passwd
Changing password for itsafe.
Current password:
New password:
Retype new password:
passwd: password updated successfully
itsafe@linuxsec:~$ █

```

Account Lockout Policy (faillock)

To mitigate **brute-force** and **password guessing** attacks, an account lockout policy was implemented using the **PAM faillock** module.

The faillock configuration file was edited:

```
sudo nano /etc/security/faillock.conf
```

The required security options were enabled by uncommenting the relevant parameters and configuring the desired lockout policy.

The configured settings define how the operating system responds to repeated authentication failures, including:

- The number of failed login attempts permitted before an account is temporarily locked.
- The duration of the lockout period.
- The time window during which failed authentication attempts are counted.

This mechanism significantly reduces the effectiveness of automated password attacks by preventing unlimited authentication attempts against user accounts.

```
# Will log the user name into the system log if the user is not found.
# Enabled if option is present.
audit
#
# Don't print informative messages.
# Enabled if option is present.
# silent
#
# Don't log informative messages via syslog.
# Enabled if option is present.
# no_log_info
#
# Only track failed user authentication attempts for local users
# in /etc/passwd and ignore centralized (AD, IdM, LDAP, etc.) users.
# The 'faillock' command will also no longer track user failed
# authentication attempts. Enabling this option will prevent a
# double-lockout scenario where a user is locked out locally and
# in the centralized mechanism.
# Enabled if option is present.
# local_users_only
#
# Deny access if the number of consecutive authentication failures
# for this user during the recent interval exceeds n tries.
# The default is 3.
deny = 3
#
# The length of the interval during which the consecutive
# authentication failures must happen for the user account
# lock out is <replaceable>n</replaceable> seconds.
# The default is 900 (15 minutes).
# fail_interval = 900
#
# The access will be re-enabled after n seconds after the lock out.
# The value 0 has the same meaning as value 'never' - the access
# will not be re-enabled without resetting the faillock
# entries by the 'faillock' command.
# The default is 600 (10 minutes).
unlock_time = 600
#
```

The account lockout policy was configured to lock a user account after **three consecutive failed authentication attempts**. Once the threshold is reached, the account remains locked for **10 minutes**, and the failed login attempts are recorded in the system logs for auditing and security monitoring purposes.

To activate the policy within the PAM authentication stack, the following file was modified:
`sudo nano /etc/pam.d/common-auth`

Three **pam_faillock** directives were added to integrate account lockout protection into the authentication process.

Pre-authentication Check

The following directive was placed **before** the `pam_unix.so` module:

```
auth required pam_faillock.so preauth
```

This rule checks whether the account is already locked **before** password verification begins. If the account is currently locked, the authentication request is immediately rejected without validating the supplied password.

Failed Authentication Tracking

The following directive was placed **after** the `pam_unix.so` module:

```
auth required pam_faillock.so authfail
```

If authentication fails, the failed attempt counter is incremented. Once the configured threshold of three failed attempts is reached, the account is automatically locked for ten minutes.

Successful Authentication Reset

The following directive was added after the `authfail` rule:

```
auth sufficient pam_faillock.so authsucc
```

When a user successfully authenticates, the accumulated failure counter is reset. This prevents occasional mistyped passwords from eventually triggering an account lockout.

Security Benefit

Implementing **pam_faillock** significantly reduces the risk of **brute-force** and **password spraying** attacks by limiting the number of consecutive authentication attempts allowed for each user account. The generated log entries also provide valuable audit information that can be used to detect and investigate suspicious authentication activity.

```
# and should contain a list of the authentication modules that define
# the central authentication scheme for use on the system
# (e.g., /etc/shadow, LDAP, Kerberos, etc.). The default is to use the
# traditional Unix authentication mechanisms.
#
# As of pam 1.0.1-6, this file is managed by pam-auth-update by default.
# To take advantage of this, it is recommended that you configure any
# local modules either before or after the default block, and use
# pam-auth-update to manage selection of other modules. See
# pam-auth-update(8) for details.

# here are the per-package modules (the "Primary" block)
auth      required          pam_faillock.so preauth
auth      [success=1 default=ignore] pam_unix.so nullok
auth      [default=die]      pam_faillock.so authfail
auth      sufficient        pam_faillock.so authsucc
# here's the fallback if no module succeeds
auth      requisite         pam_deny.so
# prime the stack with a positive return value if there isn't one already;
# this avoids us returning an error just because nothing sets a success code
# since the modules above will each just jump around
auth      required          pam_permit.so
# and here are more per-package modules (the "Additional" block)
auth      optional         pam_cap.so
# end of pam-auth-update config
```

Account Validation

To complete the account lockout implementation, the PAM account management configuration was updated.

The following file was edited:

```
sudo nano /etc/pam.d/common-account
```

The following directive was added before the `pam_unix.so` module:

```
account required pam_faillock.so
```

This directive ensures that the account lockout policy is enforced during the account validation phase of the PAM authentication process. If a user account has been locked due to repeated failed authentication attempts, access is denied even if the correct password is provided.

Adding this rule guarantees that the configured lockout policy is consistently enforced across all authentication attempts, preventing locked accounts from bypassing the restrictions imposed by the `pam_faillock` module.

```
# here are the per-package modules (the "Primary" block)
account required          pam_faillock.so
account [success=1 new_authtok_reqd=done default=ignore] pam_unix.so
# here's the fallback if no module succeeds
```

The added directive performs account lockout verification during the account authorization phase, in addition to the authentication phase. This ensures that a locked account cannot bypass the security policy, even if valid credentials are presented.

After completing the configuration, the policy was validated using a dedicated test account (`testlock`).

The verification confirmed that:

- The account was automatically locked after three consecutive failed login attempts.
- The lockout period of 10 minutes was enforced as configured.
- Authentication attempts were denied while the account remained locked.
- Failed authentication events were successfully recorded in the system logs.

These results confirmed that the `pam_faillock` configuration was functioning correctly and providing effective protection against brute-force authentication attempts.

```
itsafe@linuxsec:~$ sudo adduser testlock
info: Adding user 'testlock' ...
info: Selecting UID/GID from range 1000 to 59999 ...
info: Adding new group 'testlock' (1001) ...
info: Adding new user 'testlock' (1001) with group 'testlock (1001)' ...
info: Creating home directory '/home/testlock' ...
info: Copying files from '/etc/skel' ...
New password:
BAD PASSWORD: The password contains less than 1 lowercase letters
Retype new password:
Sorry, passwords do not match.
New password:
Retype new password:
passwd: password updated successfully
Changing the user information for testlock
Enter the new value, or press ENTER for the default
    Full Name []:
    Room Number []:
    Work Phone []:
    Home Phone []:
    Other []:
Is the information correct? [Y/n] Y
info: Adding new user 'testlock' to supplemental / extra groups 'users' ...
info: Adding user 'testlock' to group 'users' ...
itsafe@linuxsec:~$ su testlock
Password:
su: Permission denied
itsafe@linuxsec:~$ su testlock
Password:
su: Permission denied
itsafe@linuxsec:~$ su testlock
Password:
su: Permission denied
itsafe@linuxsec:~$ su testlock
The account is locked due to 3 failed logins.
(10 minutes left to unlock)
Password:
su: Authentication failure
itsafe@linuxsec:~$ sudo faillock --user testlock
testlock:
When      Type  Source                                     Valid
2026-03-01 06:14:32 TTY    /dev/pts/2                               V
2026-03-01 06:14:43 TTY    /dev/pts/2                               V
2026-03-01 06:14:49 TTY    /dev/pts/2                               V
itsafe@linuxsec:~$
```

Password History Enforcement

To prevent users from reusing previously used passwords, the **PAM password history** module was configured.

The following configuration file was edited:

```
sudo nano /etc/pam.d/common-password
```

The following directive was added **before** the `pam_unix.so` module:

```
password required pam_pwhistory.so remember=5 use_authtok
```

This configuration enables the **pam_pwhistory** module, which enforces password history validation during password changes.

The configured options provide the following functionality:

- **required** – The password history check must succeed before the password change is accepted.
- **remember=5** – Stores the user's five most recent passwords and prevents them from being reused.
- **use_authtok** – Reuses the password token that has already passed previous PAM validation modules, ensuring a consistent and secure password change process.

By enforcing password history, the system prevents users from cycling between a small number of passwords, thereby reducing the risk of password reuse and strengthening the organization's overall password policy.

Verification

The configuration was successfully tested by attempting to reuse one of the previously stored passwords. As expected, the password change was rejected, confirming that the **pam_pwhistory** module was functioning correctly and enforcing the configured password history policy.

```
# Explanation of pam_unix options:
# The "yescrypt" option enables
#hashed passwords using the yescrypt algorithm, introduced in Debian
#11. Without this option, the default is Unix crypt. Prior releases
#used the option "sha512"; if a shadow password hash will be shared
#between Debian 11 and older releases replace "yescrypt" with "sha512"
#for compatibility . The "obscure" option replaces the old
#`OBSOLETE_CHECKS_ENAB' option in login.defs. See the pam_unix manpage
#for other options.

# As of pam 1.0.1-6, this file is managed by pam-auth-update by default.
# To take advantage of this, it is recommended that you configure any
# local modules either before or after the default block, and use
# pam-auth-update to manage selection of other modules. See
# pam-auth-update(8) for details.

# here are the per-package modules (the "Primary" block)
password            requisite                                pam_pwquality.so retry=3
password required pam_pwhistory.so remember=5 use_authtok
password            [success=1 default=ignore] pam_unix.so obscure use_authtok
# here's the fallback if no module succeeds
password            requisite                                pam_deny.so
# prime the stack with a positive return value if there isn't one already;
# this avoids us returning an error just because nothing sets a success code
# since the modules above will each just jump around
password            required                                pam_permit.so
# and here are more per-package modules (the "Additional" block)
# end of pam-auth-update config
```

Checking :

```
itsafe@linuxsec:~$ passwd
Changing password for itsafe.
Current password:
New password:
Retype new password:
Password has been already used. Choose another.
passwd: Have exhausted maximum number of retries for service
passwd: password unchanged
itsafe@linuxsec:~$
```

Password Expiration Policy

To reduce the risk associated with long-term password usage, a password expiration policy was configured for the user account.

The current password aging settings were first reviewed using:

```
sudo chage -l itsafe
```

The following password policy was then applied:

```
sudo chage -M 90 itsafe
```

Sets the **maximum password lifetime** to **90 days**, requiring the user to change the password after this period.

```
sudo chage -m 1 itsafe
```

Sets the **minimum password age** to **one day**, preventing users from changing their password multiple times in quick succession to bypass the password history policy.

```
sudo chage -W 7 itsafe
```

Configures the system to display a **warning seven days before password expiration**, allowing users sufficient time to update their credentials before access is affected.

Verification

After applying the configuration, the password aging policy was verified by running the following command again:

```
sudo chage -l itsafe
```

The output confirmed that the configured policy had been successfully applied, including:

- Maximum password lifetime of **90 days**.
- Minimum password age of **1 day**.
- Password expiration warning issued **7 days** before expiration.

Implementing a password aging policy reduces the likelihood of long-term credential exposure and encourages periodic password rotation in accordance with common enterprise security practice

```
itsafe@linuxsec:~$ sudo chage -l itsafe
Last password change           : Mar 01, 2026
Password expires               : never
Password inactive              : never
Account expires                : never
Minimum number of days between password change : 0
Maximum number of days between password change : 99999
Number of days of warning before password expires : 7
itsafe@linuxsec:~$
```

```
itsafe@linuxsec:~$ sudo chage -l itsafe
Last password change           : Mar 01, 2026
Password expires               : May 30, 2026
Password inactive              : never
Account expires                : never
Minimum number of days between password change : 1
Maximum number of days between password change : 90
Number of days of warning before password expires : 7
itsafe@linuxsec:~$
```

Account Inactivity Lock

As an additional security measure, an **account inactivity policy** was configured to automatically disable inactive accounts after password expiration.

The following command was executed:

```
sudo chage -l 30 itsafe
```

This setting defines an **inactive password period** of **30 days**. If a user's password expires and is not changed within this period, the account is automatically disabled until it is reactivated by a system administrator.

This mechanism helps reduce the risk of abandoned or forgotten accounts remaining available for unauthorized access, particularly in enterprise environments where inactive accounts may otherwise persist for extended periods.

Verification

The account aging configuration was verified using:

```
sudo chage -l itsafe
```

The output confirmed that the **Inactive** field was set to **30 days**, indicating that the account would be automatically locked if the password remained expired beyond the configured grace period.

Sudo Hardening and Privileged Access Control

The next phase of the hardening process focused on securing **sudo**, the primary privilege escalation mechanism in Linux systems.

The sudo configuration was edited using the **visudo** utility:

```
sudo visudo
```

The **visudo** command provides a safe method for editing the sudoers configuration file by performing syntax validation before saving any changes. This helps prevent configuration errors that could unintentionally block administrative access to the system.

The objective of this phase was to strengthen privileged access controls by restricting sudo usage to authorized users and enforcing secure administrative practices.

```
# User privilege specification
root    ALL=(ALL:ALL) ALL

# Members of the admin group may gain root privileges
#%admin  ALL=(ALL) ALL

# Allow members of group sudo to execute any command
#%sudo   ALL=(ALL:ALL) ALL
itsafe   ALL=(ALL:ALL) ALL
Defaults:itsafe timestamp_timeout=5
Defaults:itsafe logfile="/var/log/sudo.log"
# See sudoers(5) for more information on "@include" directives:
```

Least Privilege Configuration

To further strengthen privileged access control, the default group-based administrative privileges (%sudo and %admin) were removed from the **sudoers** configuration.

Instead of granting administrative rights through broad group membership, **sudo** privileges were explicitly assigned to a single dedicated administrative account (itsafe), following the **Principle of Least Privilege**.

This approach minimizes the attack surface by ensuring that only authorized users are permitted to perform privileged administrative operations.

In addition, the following security enhancements were configured:

- **Authentication timeout:** The sudo credential cache was limited to **5 minutes**, requiring users to re-authenticate after the timeout period before executing additional privileged commands.
- **Command logging:** Dedicated logging for **sudo** activity was enabled to improve accountability, auditing, and forensic analysis of privileged operations.

Verification

The configuration was validated using two separate user accounts:

- **testlock** – A standard user account without sudo privileges. Verification confirmed that administrative commands were correctly denied.
- **itsafe** – The designated administrative account. Verification confirmed that authorized sudo access functioned as expected and that privileged commands executed successfully.

The successful verification demonstrated that administrative privileges were correctly restricted in accordance with the **Principle of Least Privilege**, while maintaining secure and fully functional privileged access for the designated administrator account.

```
testlock@linuxsec:/home/itsafe$ sudo whoami
[sudo] password for testlock:
testlock is not in the sudoers file.
testlock@linuxsec:/home/itsafe$ exit
exit
itsafe@linuxsec:~$ sudo whoami
root
itsafe@linuxsec:~$
```

Check: UID 0

Verification of UID 0 Accounts

As a final privileged access verification step, the system was inspected to ensure that no unauthorized accounts possessed **UID 0** (superuser privileges).

The following command was executed:

```
awk -F: '($3 == 0) { print $1 }' /etc/passwd
```

This command examines the `/etc/passwd` file and lists all user accounts whose **User ID (UID)** is set to **0**. In Linux, **UID 0** represents the superuser account and grants unrestricted administrative privileges, regardless of the account name.

Verification Results

The verification confirmed that **root** was the only account assigned **UID 0**, and no additional privileged accounts were found.

Performing this validation helps identify unauthorized or accidentally created privileged accounts, reducing the risk of privilege escalation and maintaining the integrity of the system's administrative access controls.

```
itsafe@linuxsec:~$ awk -F: '($3 == 0) { print $1 }' /etc/passwd
root
itsafe@linuxsec:~$
```

Verification of Passwordless Accounts

As part of the account security assessment, the system was checked for user accounts configured without a password.

The following command was executed:

```
sudo awk -F: '($2 == "") { print $1 }' /etc/shadow
```

This command examines the /etc/shadow file and identifies user accounts whose password field is empty. An empty password field may allow authentication without a password, depending on the system's authentication configuration, and therefore represents a significant security risk.

Verification Results

The verification confirmed that **no user accounts were configured with empty password fields**. All local accounts required password-based authentication in accordance with the configured security policy.

This validation helps ensure that the system does not contain unintentionally unsecured accounts and supports compliance with Linux security best practices and enterprise hardening guidelines.

```
itsafe@linuxsec:~$ sudo awk -F: '($2 == "") { print $1 }' /etc/shadow
itsafe@linuxsec:~$
```

SSH Hardening and Secure Remote Access

To improve the security of remote administration, the SSH authentication mechanism was hardened by replacing traditional password-based authentication with **public key authentication**.

As the first step, a modern **Ed25519** key pair was generated using the following command:

```
ssh-keygen -t ed25519 -C "linuxsec-desktop"
```

The **Ed25519** algorithm was selected because it provides strong cryptographic security while offering better performance and smaller key sizes compared to traditional RSA keys. It is widely recommended for modern SSH deployments and is supported by current OpenSSH releases.

The generated key pair consists of:

- **Private Key** – Stored securely on the client system and never shared.
- **Public Key** – Copied to the remote server to enable secure, passwordless authentication.

Using SSH key-based authentication significantly reduces the risk of brute-force attacks, password guessing, and credential theft associated with password-based logins.

```
linuxsec@linuxsec:~/Desktop$ ssh-keygen -t ed25519 -C "linuxsec-desktop"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/home/linuxsec/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/linuxsec/.ssh/id_ed25519
Your public key has been saved in /home/linuxsec/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:pk0BaY5FH3q6BWR46rYah9HTqc+pNuAZ2sGLm8yfXrM linuxsec-desktop
The key's randomart image is:
+--[ED25519 256]--+
|  .+ .              |
|  .+. = .           |
|   oB +             |
|  ..* = .           |
|.o+ = o S           |
|.+=o + o            |
|===+.oo             |
|**oo.o.             |
|+*+*+E              |
+----[SHA256]-----+
linuxsec@linuxsec:~/Desktop$
```

I copy the key :
ssh-copy-id itsafe@192.168.30.151

```
linuxsec@linuxsec:~/Desktop$ ssh-copy-id itsafe@192.168.30.151
The authenticity of host '192.168.30.151 (192.168.30.151)' can't be established.
ED25519 key fingerprint is SHA256:QWfc7KUR7LDze0PjNq/7AcnFj6rdfuNpBAZ+jz7atLE.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install the new keys
itsafe@192.168.30.151's password:

Number of key(s) added: 1

Now try logging into the machine, with:  "ssh 'itsafe@192.168.30.151'"
and check to make sure that only the key(s) you wanted were added.
```

Auth without pass

```
linuxsec@linuxsec:~/Desktop$ ssh itsafe@192.168.30.151
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-101-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sun Mar  1 09:12:57 AM UTC 2026

System load:  0.0               Processes:    227
Usage of /:   26.7% of 9.75GB   Users logged in: 1
Memory usage: 4%               IPv4 address for ens33: 192.168.30.151
Swap usage:  0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Sun Mar  1 09:10:53 2026 from 192.168.30.1
itsafe@linuxsec:~$
```

SSH Server Configuration Hardening

After generating the SSH key pair, the next phase focused on hardening the **OpenSSH Server** configuration on the secured **itsafe** system.

Before making any modifications, the original SSH server configuration file was backed up to allow quick recovery if needed.

The following command was executed:

```
sudo cp /etc/ssh/sshd_config /etc/ssh/sshd_config.bak
```

Creating a backup of the original configuration is considered a best practice before modifying critical system services, ensuring that the previous configuration can be restored if a configuration error prevents remote access.

The SSH server configuration file was then opened for editing:

```
sudo nano /etc/ssh/sshd_config
```

The following security settings were applied to strengthen the SSH service and reduce the attack surface of the server. Each configuration option was verified after implementation to ensure secure and reliable remote administration.

```
# To disable tunneled clear text passwords
PasswordAuthentication no
PermitEmptyPasswords no

# Change to yes to enable challenge-response authentication
# (some PAM modules and threads)
KbdInteractiveAuthentication no

# Kerberos options
#KerberosAuthentication no
#KerberosOrLocalPasswd yes

# Logging
#SyslogFacility AUTH
#LogLevel INFO

# Authentication:
LoginGraceTime 30s
PermitRootLogin no
#StrictModes yes
MaxAuthTries 3
MaxSessions 2

PubkeyAuthentication yes

# Expect .ssh/authorized_keys2
#AuthorizedKeysFile .ssh/authorized_keys

# and KbdInteractiveAuthentication
UsePAM yes

#AllowAgentForwarding yes
#AllowTcpForwarding yes
#GatewayPorts no
X11Forwarding yes
#X11DisplayOffset 10
#X11UseLocalhost yes
#PermitTTY yes
PrintMotd no
#PrintLastLog yes
#TCPKeepAlive yes
#PermitUserEnvironment no
#Compression delayed
ClientAliveInterval 300
ClientAliveCountMax 0
#UseDNS no
#PidFile /run/sshd.pid
#MaxStartups 10:30:100
#PermitTunnel no
#ChrootDirectory none
#VersionAddendum none

# no default banner path
Banner no
```

SSH Security Configuration

The OpenSSH server was hardened by disabling password-based authentication, preventing direct root access, restricting authentication attempts and concurrent sessions, limiting

access to authorized users, terminating inactive connections, and disabling unnecessary SSH features.

These measures significantly reduce the server's attack surface and provide protection against **brute-force attacks**, unauthorized remote access, and SSH service abuse.

Root Login Protection

PermitRootLogin no

Disables direct SSH login for the **root** account, forcing administrators to authenticate using a standard user account before elevating privileges with **sudo**. This reduces the risk of attacks targeting the most privileged account on the system.

Disable Password Authentication

PasswordAuthentication no

Disables password-based authentication, eliminating the primary attack vector used in brute-force and password guessing attacks.

Disable Keyboard-Interactive Authentication

KbdInteractiveAuthentication no

Disables alternative interactive authentication methods that may still rely on passwords or PAM-based prompts, ensuring that only public key authentication is accepted.

Enable Public Key Authentication

PubkeyAuthentication yes

Allows authentication exclusively through cryptographic SSH key pairs, providing significantly stronger security than traditional passwords.

Authorized Keys

The server authenticates users using the **authorized_keys** mechanism, allowing precise control over which public keys are permitted to access the system.

Restrict SSH Access to Authorized Users

AllowUsers itsafe

Limits SSH access exclusively to the designated administrative account (itsafe), preventing unauthorized local accounts from attempting remote logins.

Disable X11 Forwarding

X11Forwarding no

Disables graphical X11 forwarding, reducing unnecessary functionality and eliminating a potential attack vector.

Disable TCP Forwarding

AllowTcpForwarding no

Prevents SSH from being used to create TCP tunnels or port-forwarding channels that could bypass network security controls.

Disable SSH Tunneling

PermitTunnel no

Prevents the creation of VPN-style network tunnels through the SSH service, further reducing opportunities for unauthorized network pivoting.

Limit Authentication Attempts

MaxAuthTries 3

Restricts each connection to a maximum of **three authentication attempts**, reducing the effectiveness of brute-force attacks.

Reduce Login Grace Period

LoginGraceTime 30

Limits unauthenticated SSH sessions to **30 seconds**. Connections that are not successfully authenticated within this period are automatically terminated.

Limit Concurrent Sessions

MaxSessions 2

Restricts each authenticated SSH connection to a maximum of **two concurrent sessions**, reducing the potential impact of session abuse while still allowing legitimate administrative activity.

Security Impact

Collectively, these configuration changes transform the SSH service from its default installation into a significantly more secure remote administration platform. By enforcing **public key authentication**, eliminating unnecessary SSH capabilities, restricting administrative access, and limiting authentication attempts, the overall attack surface is substantially reduced while maintaining secure and manageable remote access for authorized administrators.

ClientAliveInterval 300

Session Timeout

ClientAliveInterval 300

Configures the SSH server to send a keepalive message every **300 seconds (5 minutes)** to verify that the client is still responsive.

ClientAliveCountMax 0

Terminates the SSH session immediately if the client fails to respond to the first keepalive message. This helps prevent abandoned or inactive SSH sessions from remaining open indefinitely.

PAM Integration

UsePAM yes

Keeps **PAM (Pluggable Authentication Modules)** enabled, ensuring that all configured account security policies—including password complexity, account lockout, password aging, and other authentication controls—remain enforced for SSH logins.

Ubuntu 24.04.3 Configuration Override

During the hardening process, it was identified that **Ubuntu Server 24.04.3** loads additional SSH configuration files from the following directory:

```
/etc/ssh/sshd_config.d/
```

One of these files, **50-cloud-init.conf**, may override settings defined in the primary `sshd_config` file, particularly on systems originally prepared for cloud deployment.

Because this project was deployed in a local lab environment rather than a cloud infrastructure, the Cloud-Init configuration was disabled to ensure that the custom hardening settings remained fully effective.

The following command was executed:

```
sudo mv /etc/ssh/sshd_config.d/50-cloud-init.conf \  
/etc/ssh/sshd_config.d/50-cloud-init.conf.disabled
```

This prevented the Cloud-Init configuration from overriding the manually configured SSH security settings.

Service Restart

After completing the configuration changes, the OpenSSH service was restarted to apply the new settings.

Security Validation

To verify that password authentication had been successfully disabled, a connection test was performed from the **Ubuntu Desktop** client.

The following command intentionally forced the SSH client to attempt **password authentication only**, bypassing public key authentication:

```
ssh -o PreferredAuthentications=password itsafe@192.168.30.151
```

The authentication attempt was **correctly rejected**, confirming that password-based authentication had been disabled and that the server accepted **public key authentication only**, as defined in the hardening policy.

This verification confirmed that the SSH hardening configuration was successfully implemented and effectively protected the server against password-based remote attacks while maintaining secure administrative access through cryptographic key authentication.

```
Connection to 192.168.30.151 closed.  
linuxsec@linuxsec:~/Desktop$ ssh -o PreferredAuthentications=password itsafe@192.168.30.151  
itsafe@192.168.30.151: Permission denied (publickey).
```

Fail2Ban

Intrusion Prevention with Fail2Ban

To provide automated protection against brute-force attacks, **Fail2Ban** was installed and configured to monitor SSH authentication logs and dynamically block malicious IP addresses.

The package was installed using:

```
sudo apt install fail2ban -y
```

A local configuration file was then created:

```
sudo nano /etc/fail2ban/jail.local
```

Using a local configuration file (jail.local) preserves the default configuration files and follows the recommended Fail2Ban deployment methodology.

SSH Jail Configuration

A dedicated **SSH jail** was configured with the following security policy:

- **Monitored Service:** OpenSSH
- **Log Backend:** systemd
- **Detection Threshold:** 3 failed authentication attempts
- **Detection Window:** 10 minutes
- **Ban Duration:** 1 hour

This configuration enables Fail2Ban to automatically identify repeated failed authentication attempts originating from the same IP address and temporarily block further access using the system firewall.

Validation

To verify the configuration, a practical attack simulation was performed from the **Ubuntu Desktop** client.

Since SSH password authentication had previously been disabled as part of the hardening process, password authentication was **temporarily re-enabled** solely for testing purposes. This allowed multiple failed login attempts to be generated against the SSH service.

Three consecutive failed authentication attempts were then initiated from the Ubuntu Desktop system.

The protected Ubuntu Server successfully detected the repeated failures and automatically added the client's IP address to the Fail2Ban ban list.

The test confirmed that:

- Failed authentication attempts were successfully detected.
- The configured threshold of **three failed logins** was correctly enforced.
- The attacking IP address was automatically blocked.
- The ban was applied for the configured duration of **one hour**.

Following the successful verification, the temporary testing configuration was removed and the hardened SSH configuration was restored, ensuring that the server once again accepted **public key authentication only**.

Security Benefit

Fail2Ban provides an additional defensive layer by automatically responding to repeated authentication failures. Combined with **public key authentication**, **pam_faillock**, and the hardened SSH configuration implemented earlier in this project, it significantly reduces the effectiveness of brute-force attacks and unauthorized remote access attempts.

```
Executing: /usr/lib/systemd/systemd-sysv-install enable fail2ban
● fail2ban.service - Fail2Ban Service
   Loaded: loaded (/usr/lib/systemd/system/fail2ban.service; enabled; preset:➤
   Active: active (running) since Sun 2026-03-01 14:54:11 UTC; 11s ago
     Docs: man:fail2ban(1)
  Main PID: 15863 (fail2ban-server)
    Tasks: 5 (limit: 9379)
   Memory: 24.9M (peak: 25.4M)
      CPU: 223ms
   CGroup: /system.slice/fail2ban.service
           └─15863 /usr/bin/python3 /usr/bin/fail2ban-server -xf start
```

```
GNU nano 7.2
[sshd]
enabled = true
port = 22
filter = sshd
logpath = /var/log/auth.log
maxretry = 3
findtime = 10m
bantime = 1h
backend = systemd
```

```
itsafe@linuxsec:~$ sudo systemctl restart fail2ban
itsafe@linuxsec:~$ sudo fail2ban-client status
Status
|- Number of jail:      1
`- Jail list:  sshd
itsafe@linuxsec:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
| |- Currently failed: 0
| |- Total failed:     0
| `-- Journal matches: _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
  |- Currently banned: 0
  |- Total banned:     0
  `-- Banned IP list:
itsafe@linuxsec:~$
```

: check

```
linuxsec@linuxsec: ~/Desktop
linuxsec@linuxsec:~/Desktop$ ssh fakeuser@192.168.30.151
fakeuser@192.168.30.151's password:
Permission denied, please try again.
fakeuser@192.168.30.151's password:
Received disconnect from 192.168.30.151 port 22:2: Too many authentication failures
Disconnected from 192.168.30.151 port 22
linuxsec@linuxsec:~/Desktop$
```

```
itsafe@linuxsec:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
| |- Currently failed: 2
| |- Total failed:     5
| `-- Journal matches: _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
  |- Currently banned: 1
  |- Total banned:     1
  `-- Banned IP list:  192.168.30.148
itsafe@linuxsec:~$
```

ufw

Network Access Control with UFW

As part of the network hardening process, **UFW (Uncomplicated Firewall)** was configured to restrict SSH access to trusted hosts only.

The VMware NAT gateway was explicitly allowed to establish SSH connections:

```
sudo ufw allow from 192.168.30.1 to any port 22 proto tcp
```

In addition, the **Ubuntu Desktop** workstation, used as the administrative client, was granted SSH access:

```
sudo ufw allow from 192.168.30.148 to any port 22 proto tcp
```

These rules ensure that only trusted management systems are permitted to establish remote administrative connections to the protected server.

Verification

Connection attempts were performed from an IP address that was **not included** in the firewall allow-list.

The test confirmed that unauthorized hosts were unable to establish an SSH connection to **192.168.30.151**, while the explicitly authorized hosts continued to connect successfully.

This validation confirmed that the firewall rules were functioning correctly and enforcing the intended network access policy.

Firewall Hardening with iptables

To better understand Linux packet filtering at the kernel level, the firewall configuration was recreated using **iptables**, implementing the same security policy previously enforced with UFW.

Before applying the new rules, the existing firewall configuration was cleared and verified to ensure that no previous rules remained active.

Default Policy

The following default security policy was implemented:

```
sudo iptables -P INPUT DROP
```

```
sudo iptables -P FORWARD DROP
```

```
sudo iptables -P OUTPUT ACCEPT
```

This configuration establishes a **default-deny** security model:

- All inbound traffic is denied.
- All forwarded traffic is denied.
- Outbound traffic is permitted.

A default-deny policy significantly reduces the attack surface by allowing only explicitly authorized traffic.

Allow Established Connections

To maintain existing and related network sessions, the following rule was added:

```
sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
```

This permits packets belonging to already established or related connections while continuing to block unsolicited inbound traffic.

Allow Loopback Traffic

The local loopback interface was enabled:

```
sudo iptables -A INPUT -i lo -j ACCEPT
```

Loopback communication is required for many local services and inter-process communications running on the host.

Restrict SSH Access

SSH access was explicitly permitted only from trusted management systems.

VMware NAT Gateway:

```
sudo iptables -A INPUT -p tcp -s 192.168.30.1 --dport 22 -j ACCEPT
```

Ubuntu Desktop:

```
sudo iptables -A INPUT -p tcp -s 192.168.30.148 --dport 22 -j ACCEPT
```

No additional hosts were granted SSH access.

Validation

After applying the firewall rules, connectivity tests were performed.

The verification confirmed that:

- Authorized management hosts successfully established SSH connections.
 - Unauthorized systems were unable to access the protected server.
 - The firewall correctly enforced the defined access control policy.
-

Security Summary

The implemented firewall policy follows a **default-deny** approach by dropping all unsolicited inbound traffic while allowing only explicitly authorized communications.

The final configuration provides protection by:

- Blocking unsolicited inbound traffic.
- Allowing only **ESTABLISHED** and **RELATED** connections.
- Permitting local loopback communication.
- Restricting SSH access to trusted management hosts only.

This configuration significantly reduces the server's network attack surface while maintaining secure remote administration.

Modern Linux Firewall Architecture

It is important to note that **TCP Wrappers** are no longer relevant on modern Ubuntu systems.

Beginning with recent OpenSSH releases, the SSH daemon no longer relies on the **libwrap** library. Access control is instead enforced through:

- Firewall policies (**nftables** / **iptables**)
- OpenSSH configuration (sshd_config)
- PAM authentication modules

Although this project demonstrates firewall configuration using **iptables**, modern Linux distributions—including **Ubuntu 24.04 LTS**—use **nftables** as the underlying packet filtering framework.

On current Ubuntu releases, many iptables commands are internally translated and managed through the **nftables** compatibility layer, making **nftables** the recommended long-term firewall technology.

```
To          Action      From
--          -
22/tcp      ALLOW IN    192.168.30.148
22/tcp      ALLOW IN    192.168.30.1

itsafe@linuxsec:~$
```

```
(shlomi@kali)~$ ssh linuxsec@192.168.30.148
The authenticity of host '192.168.30.148 (192.168.30.148)' can't be established.
ED25519 key fingerprint is SHA256:Vhgc0RBS0ki7McLnsZ+BvYQ/b0RHwvNnsnvuonLLYks.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.30.148' (ED25519) to the list of known hosts.
linuxsec@192.168.30.148's password:
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.17.0-14-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

204 updates can be applied immediately.
27 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

linuxsec@linuxsec:~$ exit
logout
Connection to 192.168.30.148 closed.

(shlomi@kali)~$ ssh itsafe@192.168.30.151
ssh: connect to host 192.168.30.151 port 22: Connection timed out
```

```
itsafe@linuxsec:~$ sudo iptables -F
sudo iptables -X
sudo iptables -Z
itsafe@linuxsec:~$ sudo iptables -L -n -v
Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source                   destination

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source                   destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target    prot opt in     out     source                   destination
itsafe@linuxsec:~$
```

```

itsafe@linuxsec:~$ sudo iptables -L -n -v
Chain INPUT (policy DROP 0 packets, 0 bytes)
  pkts bytes target    prot opt in     out     source               destination
    18 2072 ACCEPT    0    --  *      *       0.0.0.0/0            0.0.0.0/0            ctstate RELATED,ESTABLISHED
     0 0 ACCEPT    0    --  lo     *       0.0.0.0/0            0.0.0.0/0
     0 0 ACCEPT    6    --  *      *       192.168.30.1         0.0.0.0/0            tcp dpt:22
     0 0 ACCEPT    6    --  *      *       192.168.30.148      0.0.0.0/0            tcp dpt:22

Chain FORWARD (policy DROP 0 packets, 0 bytes)
  pkts bytes target    prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 10 packets, 1512 bytes)
  pkts bytes target    prot opt in     out     source               destination
itsafe@linuxsec:~$

```

```

$ ip netns exec ens33
inet 192.168.30.148/24 brd 192.168.30.255 scope global dynamic noprefixroute ens33
    valid_lft 1559sec preferred_lft 1559sec
inet6 fe80::20c:29ff:fe6e:831f/64 scope link
    valid_lft forever preferred_lft forever
linuxsec@linuxsec:~/Desktop$ ssh itsafe@192.168.30.151
itsafe@192.168.30.151's password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-101-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sun Mar 1 04:22:34 PM UTC 2026

System load:  0.0               Processes:            224
Usage of /:   26.6% of 9.75GB   Users logged in:     1
Memory usage: 4%               IPv4 address for ens33: 192.168.30.151
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Sun Mar 1 16:22:35 2026 from 192.168.30.148
itsafe@linuxsec: $

```

```
itsafe@linuxsec:~$ sudo nft list ruleset
[sudo] password for itsafe:
# Warning: table ip filter is managed by iptables-nft, do not touch!
table ip filter {
    chain INPUT {
        type filter hook input priority filter; policy drop;
        ct state related,established counter packets 379 bytes 39778 accept
        iifname "lo" counter packets 0 bytes 0 accept
        ip saddr 192.168.30.1 tcp dport 22 counter packets 0 bytes 0 accept
        ip saddr 192.168.30.148 tcp dport 22 counter packets 2 bytes 120 accept
    }

    chain FORWARD {
        type filter hook forward priority filter; policy drop;
    }

    chain OUTPUT {
        type filter hook output priority filter; policy accept;
    }
}
```

Secure Storage Configuration

As part of the operating system hardening process, a dedicated **10 GB virtual disk** was added to the virtual machine to provide an isolated environment for filesystem security testing.

This dedicated storage area was used to safely evaluate Linux security mechanisms—including mount options, Sticky Bit, SUID/SGID permissions, Access Control Lists (ACLs), and File Integrity Monitoring (FIM)—without affecting the primary operating system partition.

Virtual Disk Configuration

The virtual disk was configured using the **VMware SCSI controller**, which is the recommended storage interface for virtual server environments due to its improved performance, stability, and compatibility.

After adding the virtual disk, the operating system was instructed to detect the new hardware.

The following command was used to verify that the disk had been successfully recognized:

```
lsblk
```

The verification confirmed that the operating system detected the newly attached storage device as:

```
/dev/sdb
```

At this stage, the disk contained no active partitions and was ready for secure initialization.

Full Disk Encryption with LUKS

To protect sensitive data stored on the new partition, **LUKS (Linux Unified Key Setup)** encryption was implemented.

LUKS provides full block-level encryption, ensuring that all data written to the disk remains inaccessible until the correct passphrase is supplied.

This protects the confidentiality of stored information even if the physical storage device is removed or the virtual disk is accessed outside the operating system.

Filesystem Creation

After unlocking the encrypted LUKS container, an **ext4** filesystem was created on the decrypted logical device:

```
/dev/mapper/secure_lab_crypt
```

Although applications interact with a standard Linux filesystem, all disk operations are transparently encrypted at the block layer before being written to the physical storage device.

Secure Mount Point

A dedicated mount point named:

/secure-lab

was created for the encrypted filesystem.

The encrypted partition was then mounted using additional filesystem hardening options to further reduce the attack surface.

By combining **full-disk encryption** with **secure mount options**, the solution provides both:

- Data confidentiality
- Protection against code execution and privilege escalation from the mounted filesystem

Automatic Unlock During Boot

To allow secure automatic mounting after system startup, the encrypted volume was integrated into the Linux boot process.

crypttab

The **/etc/crypttab** configuration file was updated to instruct the operating system to unlock the encrypted LUKS volume during boot and create the logical device:

secure_lab_crypt

fstab

The decrypted device:

/dev/mapper/secure_lab_crypt

was then configured inside:

/etc/fstab

to mount automatically at:

/secure-lab

using the **ext4** filesystem.

The following security mount options were configured:

Mount Option	Security Purpose
noexec	Prevents execution of binaries directly from the mounted filesystem.
nosuid	Disables SUID and SGID privilege escalation mechanisms.
nodev	Prevents interpretation of device files stored on the partition.
acl	Enables Access Control Lists (ACLs) for fine-grained permission management.

Mount Option	Security Purpose
nofail	Allows the system to continue booting normally even if the encrypted volume is unavailable.

Additional filesystem parameters were configured as follows:

- **0** – Excludes the filesystem from the legacy dump backup utility.
- **2** – Schedules filesystem consistency checks (fsck) after the root filesystem during system startup.

This configuration ensures that the encrypted partition is mounted securely while maintaining system availability and minimizing the attack surface.

Validation

To validate the configuration, the following verification procedure was performed:

1. The encrypted filesystem was unmounted.
2. The decrypted device under /dev/mapper was closed.
3. System configuration was reloaded after updating both crypttab and fstab.
4. The virtual machine was rebooted.

During system startup, the operating system correctly detected the encrypted LUKS partition and requested the configured passphrase.

After successful authentication:

- The encrypted container was unlocked.
- The logical device was automatically created.
- The filesystem was mounted automatically according to the fstab configuration.
- All configured hardening options were successfully applied.

The successful validation confirmed that encrypted storage, automatic boot integration, and secure mount configuration were functioning as expected.

Security Benefit

The implemented solution combines **full-disk encryption**, **secure boot integration**, and **filesystem hardening** to protect sensitive data while reducing the risk of privilege escalation, unauthorized code execution, and offline access to stored information. This layered approach significantly strengthens the overall security posture of the Linux server and aligns with

enterprise Linux hardening best practices.

```

itsafe@linuxsec:~$ lsblk
NAME                                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                                  8:0      0   20G  0 disk
├─sda1                              8:1      0    1M  0 part
├─sda2                              8:2      0   1.8G  0 part /boot
├─sda3                              8:3      0  18.2G  0 part
└─ubuntu--vg-ubuntu--lv 252:0    0    10G  0 lvm  /
sdb                                  8:16     0    10G  0 disk
sr0                                  11:0     1    3.1G  0 rom

itsafe@linuxsec:~$ sudo fdisk /dev/sdb
[sudo] password for itsafe:

Welcome to fdisk (util-linux 2.39.3).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table.
Created a new DOS (MBR) disklabel with disk identifier 0x4b19a077.

Command (m for help): n
Partition type
  p   primary (0 primary, 0 extended, 4 free)
  e   extended (container for logical partitions)
Select (default p): p
Partition number (1-4, default 1): 1
First sector (2048-20971519, default 2048):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-20971519, default 20971519):

Created a new partition 1 of type 'Linux' and of size 10 GiB.

Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.

itsafe@linuxsec:~$ lsblk
NAME                                MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda                                  8:0      0   20G  0 disk
├─sda1                              8:1      0    1M  0 part
├─sda2                              8:2      0   1.8G  0 part /boot
├─sda3                              8:3      0  18.2G  0 part
└─ubuntu--vg-ubuntu--lv 252:0    0    10G  0 lvm  /
sdb                                  8:16     0    10G  0 disk
├─sdb1                              8:17     0    10G  0 part
sr0                                  11:0     1    3.1G  0 rom

itsafe@linuxsec:~$

```

```

itsafe@linuxsec:~$ sudo mkfs.ext4 /dev/mapper/secure_lab_crypt
mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 2617088 4k blocks and 655360 inodes
Filesystem UUID: 878fbb3c-2a6d-4d5e-b248-6958e1514103
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

itsafe@linuxsec:~$ sudo mkdir /secure-lab
itsafe@linuxsec:~$ sudo mount -o noexec,nosuid,nodev /dev/mapper/secure_lab_crypt /secure-lab
itsafe@linuxsec:~$

```

```

itsafe@linuxsec:~$ sudo blkid /dev/sdb1
/dev/sdb1: UUID="f97d006a-dfc0-45ef-94a4-5be29dd85d00" TYPE="crypto_LUKS" PARTUUID="4b19a077-01"
itsafe@linuxsec:~$ sudo nano /etc/crypttab

```

```

GNU nano 7.2
# <target name> <source device>      <key file>      <options>
secure_lab_crypt UUID=f97d006a-dfc0-45ef-94a4-5be29dd85d00 none luks

```

```
GNU nano 7.2
# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a
# device; this may be used with UUID= as a more robust way to name devices
# that works even if disks are added and removed. See fstab(5).
#
# <file system> <mount point> <type> <options> <dump> <pass>
# / was on /dev/ubuntu-vg/ubuntu-lv during curtin installation
/dev/disk/by-id/dm-uuid-LVM-bkD8Mp0030OVZ6mpNUWTalM6TtojJONsFQD4fc2BNUelWmLFH2uJ3zGsfjMEW9KY / ext4 defaults 0 1
# /boot was on /dev/sda2 during curtin installation
/dev/disk/by-uuid/b773c193-1be1-1700-b2ec-c079a0b2120b /boot ext4 defaults 0 1
/dev/mapper/secure_lab_crypt /secure-lab ext4 defaults,noexec,nosuid,nodev,acl,nofail 0 2
```

```
itsafe@linuxsec:~$ sudo umount /secure-lab
sudo cryptsetup close secure_lab_crypt
sudo systemctl daemon-reload
sudo mount -a
itsafe@linuxsec:~$
```

```
[ OK ] Started systemd-udev.service - Kernel Manager for Device Events and Files.
[ OK ] Finished systemd-udev-trigger.service - Coldplug All udev Devices.
[ OK ] Started systemd-ask-password-console.requests to Console Directory Watch.
[ OK ] Found device dev-disk-by\x2duuid-f..d85d00.device - VMware_Virtual_S 1.
Starting systemd-cryptsetup@secure_raphy Setup for secure_lab_crypt...
Please enter passphrase for disk VMware_Virtual_S (secure_lab_crypt) on /secure-
lab: (press TAB for no echo) [ 5.298734] piix4_smbus 0000:00:07.3: SMBus Host
Controller not enabled!

Please enter passphrase for disk VMware_Virtual_S (secure_lab_crypt) on /secure-lab: (press TAB for no echo)
```

```
itsafe@linuxsec:~$ mount | grep secure-lab
ls /dev/mapper
/dev/mapper/secure_lab_crypt on /secure-lab type ext4 (rw,nosuid,nodev,noexec,relatime)
control secure_lab crypt ubuntu--vg-ubuntu--lv
itsafe@linuxsec:~$
```

Access Control Lists (ACL)

To demonstrate the capabilities of Access Control Lists (ACLs) beyond the standard Linux permission model, two dedicated test users were created:

- Alice – Authorized user with extended access.
- Bob – Restricted user with limited permissions.

The objective of this exercise was to illustrate how ACLs provide fine-grained access control, allowing permissions to be assigned to individual users without changing the file owner or the primary group.

The encrypted /secure-lab filesystem, created earlier in this project, was used as the testing environment.

Alice was granted permission to:

- Access the encrypted directory.
- Create new files.
- Modify existing files.
- Execute files stored within the directory.

Bob was assigned a more restrictive permission set in order to demonstrate how ACLs can enforce different levels of access for individual users sharing the same filesystem.

This scenario highlights one of the primary advantages of ACLs over the traditional Linux permission model: the ability to define user-specific permissions without modifying ownership or group membership, making ACLs particularly valuable in enterprise environments where multiple users require different levels of access to shared resources.

Access Control Lists (ACL)

To demonstrate the capabilities of **Access Control Lists (ACLs)**** beyond the standard Linux permission model, two dedicated test users were created:**

- ****Alice**** – Authorized user with extended access.
- ****Bob**** – Restricted user with limited permissions.

The objective of this exercise was to illustrate how ACLs provide ****fine-grained access control****, allowing permissions to be assigned to individual users without changing the file owner or the primary group.

The encrypted ****`/secure-lab`**** filesystem, created earlier in this project, was used as the testing environment.

Alice was granted permission to:

- Access the encrypted directory.
- Create new files.
- Modify existing files.
- Execute files stored within the directory.

Bob was assigned a more restrictive permission set in order to demonstrate how ACLs can enforce different levels of access for individual users sharing the same filesystem.

This scenario highlights one of the primary advantages of ACLs over the traditional Linux permission model: the ability to define ****user-specific permissions**** without modifying ownership or group membership, making ACLs particularly valuable in enterprise environments where multiple users require different levels of access to shared resources.

```

itsafe@linuxsec:~$ sudo setfacl -m u:alice:rwX /secure-lab/projects
itsafe@linuxsec:~$ sudo setfacl -m u:bob:rx /secure-lab/projects
itsafe@linuxsec:~$ getfacl /secure-lab/projects
getfacl: Removing leading '/' from absolute path names
# file: secure-lab/projects
# owner: root
# group: root
user::rwX
user:alice:rwX
user:bob:r-x
group::---
mask::rwX
other::---

itsafe@linuxsec:~$ su -alice
su: invalid option -- 'a'
Try 'su --help' for more information.
itsafe@linuxsec:~$ su - alice
Password:
$ cd /secure-lab/projects
$ touch alice.txt
$ mkdir testdir
$ ls
alice.txt  testdir
$ exit
itsafe@linuxsec:~$ su - bob
Password:
$ cd /secure-lab/projects
$ ls
alice.txt  testdir
$ touch bob.txt
touch: cannot touch 'bob.txt': Permission denied
$ █

```

ACL Verification

The configured Access Control Lists (ACLs) were successfully validated.

As shown in the screenshot above:

- **Alice** successfully accessed the encrypted directory and was able to create files and directories in accordance with the assigned ACL permissions.
- **Bob** was able to browse the directory contents but was denied permission when attempting to create a new file, resulting in the expected **"Permission denied"** message.

The successful validation confirms that the configured ACL rules were correctly enforced and that individual user permissions were applied as intended. This demonstrates how ACLs provide fine-grained access control beyond the traditional Linux permission model while maintaining the security of shared resources.

Monitoring and Detection

Following the implementation of filesystem hardening through **LUKS encryption**, secure mount options, and **Access Control Lists (ACLs)**, the next phase of the project focused on implementing an advanced monitoring and detection layer.

The objective of this stage was to transition from **passive protection (system hardening)** to **active security monitoring**. Rather than only preventing unauthorized access, the system was configured to detect unauthorized modifications, privilege abuse, policy violations, and other suspicious activities in real time.

The monitoring architecture implemented during this phase includes:

- **File Integrity Monitoring (FIM)** to detect unauthorized file modifications.
- **User activity and permission change monitoring.**
- **Continuous security logging** for audit and forensic analysis.
- **Detection of critical operating system events** at the kernel level.

Linux Audit Framework (auditd)

To provide comprehensive operating system auditing, the **Linux Audit Framework (auditd)** was installed and configured.

The `auditd` service enables **kernel-level monitoring** of security-relevant events, including file modifications, permission changes, authentication events, privilege escalation attempts, and user activity.

After installation, the service was started and configured to **launch automatically during system boot**, ensuring continuous monitoring immediately after the operating system becomes operational.

By collecting detailed audit records directly

```

itsafe@linuxsec:~$ sudo systemctl status auditd
● auditd.service - Security Auditing Service
   Loaded: loaded (/usr/lib/systemd/system/auditd.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-03-02 07:16:06 UTC; 15s ago
     Docs: man:auditd(8)
           https://github.com/linux-audit/audit-documentation
   Main PID: 2207 (auditd)
      Tasks: 2 (limit: 9376)
    Memory: 524.0K (peak: 2.2M)
       CPU: 18ms
    CGroup: /system.slice/auditd.service
            └─2207 /sbin/auditd

Mar 02 07:16:06 linuxsec augenrules[2221]: enabled 1
Mar 02 07:16:06 linuxsec augenrules[2221]: failure 1
Mar 02 07:16:06 linuxsec augenrules[2221]: pid 2207
Mar 02 07:16:06 linuxsec augenrules[2221]: rate_limit 0
Mar 02 07:16:06 linuxsec augenrules[2221]: backlog_limit 8192
Mar 02 07:16:06 linuxsec augenrules[2221]: lost 0
Mar 02 07:16:06 linuxsec augenrules[2221]: backlog 4
Mar 02 07:16:06 linuxsec augenrules[2221]: backlog_wait_time 60000
Mar 02 07:16:06 linuxsec augenrules[2221]: backlog_wait_time actual 0
Mar 02 07:16:06 linuxsec systemd[1]: Started auditd.service - Security Auditing Service.
itsafe@linuxsec:~$ █

```

```

# Monitor secure-lab directory access
-w /secure-lab -p rwa -k secure_lab_access

# Monitor permission changes
-w /bin/chmod -p x -k perm_mod
-w /bin/chown -p x -k perm_mod
-w /usr/bin/setfacl -p x -k perm_mod

# Monitor user database changes
-w /etc/passwd -p wa -k user_changes
-w /etc/shadow -p wa -k user_changes
-w /etc/group -p wa -k user_changes

# Monitor sudo usage
-w /usr/bin/sudo -p x -k sudo_usage █

```

Advanced Audit Rules

To provide comprehensive monitoring of security-critical events, a dedicated Auditd rule set was created.

The custom audit rules were stored in the following configuration file:

```

```bash
sudo nano /etc/audit/rules.d/secure-lab.rules
```

```

The following audit rules were implemented:

File Integrity Monitoring (Encrypted Directory)

```

```text
-w /secure-lab -p rwa -k secure_lab_access
```

```

This rule continuously monitors the protected `*/secure-lab` directory.

Configuration details:

- `*-w` – Watches the specified filesystem path.
- `*/secure-lab` – The protected encrypted directory.
- `*-p rwa` – Monitors read (`r`), write (`w`), and attribute changes (`a`), including permission modifications such as `chmod`, `chown`, and ACL updates.
- `*-k secure_lab_access` – Assigns the audit records to the `secure_lab_access` key, allowing efficient filtering and investigation.

Permission Modification Monitoring

```
``text
-w /bin/chmod -p x -k perm_mod
``
```

This rule records every execution of the `chmod` utility, allowing administrators to detect and investigate permission modifications performed on the system.

User Database Monitoring

```
``text
-w /etc/passwd -p wa -k user_changes
``
```

This rule monitors changes to the `/etc/passwd` file, helping detect unauthorized modifications to local user accounts.

Privileged Command Monitoring

```
``text
-w /usr/bin/sudo -p x -k sudo_usage
```

...

This rule records every execution of the **sudo** command, providing a complete audit trail of privileged administrative activity.

Validation

The monitoring configuration was validated by performing controlled security-related activities on the protected system.

The generated audit logs confirmed that:

- A permission modification performed using **chmod** was successfully detected and recorded under the **perm_mod** audit key.
- The logs correctly identified that the **itsafe** account elevated privileges through **sudo** before executing the administrative operation.
- An access attempt to the protected **/secure-lab** directory by the **bob** user was successfully recorded under the **secure_lab_access** audit key.

The verification demonstrated that the Linux Audit Framework successfully captured privileged operations, filesystem activity, and user actions directly at the **kernel level**.

Security Benefit

By combining **LUKS encryption**, **ACL-based access control**, **secure filesystem configuration**, and **Auditd kernel-level monitoring**, the system provides multiple complementary security layers.

This approach not only protects sensitive information from unauthorized access but also generates detailed forensic evidence for every critical security event, supporting incident detection, investigation, and compliance with enterprise Linux security best practices.

****ICMP Timestamp Response (Information Disclosure)****

Although classified as a low-risk informational issue, responding to ICMP Timestamp requests may provide attackers with additional information that can assist in operating system fingerprinting and reconnaissance activities.

To further reduce information disclosure, a dedicated firewall rule was implemented to block ICMP Timestamp requests, minimizing the server's exposure to network reconnaissance techniques.

Following this mitigation, the server presented a significantly reduced external attack surface while maintaining normal operational functionality.

Final Remarks

This project demonstrates not only the implementation of individual Linux security controls, but also the integration of multiple defensive technologies into a unified hardening strategy.

By combining ****system hardening****, ****network security****, ****access control****, ****encryption****, ****continuous auditing****, and ****vulnerability assessment****, the resulting server configuration reflects security practices commonly applied in enterprise Linux environments.

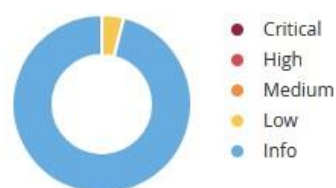
The project provided valuable hands-on experience in designing, implementing, validating, and documenting a comprehensive Linux security architecture while reinforcing industry-recognized hardening methodologies and operational best practices.

| Filter Search Vulnerabilities 8 Vulnerabilities | | | | | | |
|---|------|-----|------|---|-------------------|-------|
| Sev | CVSS | VPR | EPSS | Name | Family | Count |
| LOW | 2.1 | | | ICMP Timestamp Request Remote Date Disclosure | General | 1 |
| INFO | ... | ... | ... | SSH (Multiple Issues) | Misc. | 2 |
| INFO | ... | ... | ... | SSH (Multiple Issues) | Service detection | 2 |

Scan Details

| | |
|----------------|---|
| Policy: | Advanced Scan |
| Status: | Completed |
| Severity Base: | CVSS v3.0  |
| Scanner: | Local Scanner |
| Start: | Today at 4:58 PM |
| End: | Today at 4:59 PM |
| Elapsed: | 2 minutes |

Vulnerabilities



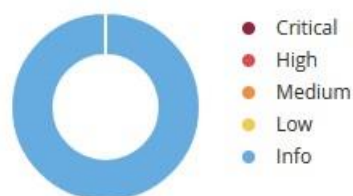
```
itsafe@linuxsec:~$ sudo iptables -A INPUT -p icmp --icmp-type timestamp-request -j DROP
sudo iptables -A OUTPUT -p icmp --icmp-type timestamp-reply -j DROP
[sudo] password for itsafe:
```

After the commend:

Scan Details

| | |
|----------------|---|
| Policy: | Advanced Scan |
| Status: | Completed |
| Severity Base: | CVSS v3.0  |
| Scanner: | Local Scanner |
| Start: | Today at 5:07 PM |
| End: | Today at 5:08 PM |
| Elapsed: | 2 minutes |

Vulnerabilities



Shlomi green